Lab Manual: Tower of Hanoi using State Space Search

Title

Implementation of Tower of Hanoi using State Space Search (Recursion) in Python

Aim

To implement the Tower of Hanoi problem using recursion in Python and understand its application in State Space Search.

Theory

The Tower of Hanoi is a classical mathematical puzzle and one of the most popular problems used to demonstrate the concepts of Recursion, State Space Search, and Divide-and-Conquer in Artificial Intelligence. The puzzle was invented by the French mathematician Édouard Lucas in 1883.

The puzzle consists of three pegs: Source Peg (S), Helper Peg (H), and Destination Peg (D), and a number of disks of different sizes. Initially, all disks are stacked on the source peg in decreasing order of size.

Rules:
1. Only one disk can be moved at a time.
2. Only the top disk of any peg can be moved.
3. A larger disk cannot be placed on top of a smaller disk.

The problem is solved recursively:
• Move (n−1) disks from Source to Helper.
• Move the largest disk from Source to Destination.
• Move (n−1) disks from Helper to Destination.

In Artificial Intelligence, Tower of Hanoi is considered a State Space Search Problem where each arrangement of disks is a state, disk movement is an action, the starting arrangement is the initial state, and the final arrangement is the goal state.

Minimum number of moves = $2^n - 1$

Time Complexity: $O(2^n)$
Space Complexity: $O(n)$

Algorithm

1. Start.
2. If n = 1, move the disk from Source to Destination and return.
3. Move n−1 disks from Source to Helper.
4. Move the nth disk from Source to Destination.
5. Move n−1 disks from Helper to Destination.
6. Repeat recursively until all disks are transferred.
7. Stop.

Real-Time Applications of Tower of Hanoi in Artificial Intelligence

1. State Space Search
2. Automated Planning Systems
3. Robotics and Motion Planning
4. Heuristic Search Techniques
5. Intelligent Decision-Making
6. Knowledge Representation
7. Expert Systems
8. Game AI Development
9. Optimization Problems
10. Machine Learning Research

Python Program

```
def tower_hanoi(n, a, b, c):
  if n == 1:
    print("Move disk 1 from", a, "to", c)
    return

  tower_hanoi(n - 1, a, c, b)
  print("Move disk", n, "from", a, "to", c)
  tower_hanoi(n - 1, b, a, c)

tower_hanoi(3, "S", "H", "D")
```

Sample Output

```
Move disk 1 from S to D
Move disk 2 from S to H
Move disk 1 from D to H
Move disk 3 from S to D
Move disk 1 from H to S
Move disk 2 from H to D
Move disk 1 from S to D
```

Learning Outcomes

1. Understand recursion.
2. Apply State Space Search techniques.
3. Implement recursive algorithms in Python.
4. Analyze complexity of recursive solutions.
5. Understand AI planning and problem-solving approaches.

Viva Questions

1. What is Tower of Hanoi?
2. What are its rules?
3. What is recursion?
4. Why is it a State Space Search problem?
5. What is the minimum number of moves required?
6. What is its time complexity?
7. What is the role of helper peg?
8. What is the base condition?
9. How is it related to AI?
10. What are its applications?

Result

The Tower of Hanoi problem was successfully implemented using recursion in Python. The experiment demonstrated recursion, state space search, and AI problem-solving techniques.